In [ ]:

```
import torch
import torchvision
from torchvision import transforms, datasets
from torch import nn, optim
from torchvision.models import mobilenet_v2, MobileNet_V2_Weights
from google.colab import files
import zipfile, os
import matplotlib.pyplot as plt
from PIL import Image

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# --- 1. Dataset Extraction ---
# Make sure to upload your dataset.zip when prompted
print("Upload dataset.zip:")
uploaded = files.upload()

for file in uploaded.keys():
    with zipfile.ZipFile(file, 'r') as zip_ref:
        zip_ref.extractall("/content/")

# --- 2. Advanced Augmentation ---
# This forces the model to learn features deeply, leading to higher confidence
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]
])

val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]
])

# --- 3. Load Data ---
# Path updated to match your zip structure: /content/dataset/train
train_path = "/content/dataset/train"
train_data = datasets.ImageFolder(train_path, transform=transform)
train_loader = torch.utils.data.DataLoader(train_data, batch_size=8, shuffle=T

# Important: Mapping classes
# index 0: elephant, index 1: not_elephant
print(f"Detected Classes: {train_data.class_to_idx}")

# --- 4. Model fine-tuning ---
model = mobilenet_v2(weights=MobileNet_V2_Weights.DEFAULT)

# Freeze most layers, but unfreeze the last block for high-certainty learning
for param in model.features.parameters():
```

```
    param.requires_grad = False
for param in model.features[14:].parameters():
    param.requires_grad = True

model.classifier[1] = nn.Linear(model.last_channel, 1)
model = model.to(device)

criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(model.parameters(), lr=0.0001)

# --- 5. Training ---
epochs = 15
print("\nTraining for high confidence...")

for epoch in range(epochs):
    model.train()
    running_loss = 0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.float().unsqueeze(1).to(dev

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    print(f"Epoch {epoch+1}/{epochs} | Loss: {running_loss/len(train_loader):.

# --- 6. Prediction with Specific Confidence Logic ---
print("\nUpload test image:")
test_upload = files.upload()

model.eval()
for file in test_upload.keys():
    img = Image.open(file).convert("RGB")
    img_tensor = val_transform(img).unsqueeze(0).to(device)

    with torch.no_grad():
        output = model(img_tensor)
        # Probability of being 'not_elephant' (class 1)
        prob_class_1 = torch.sigmoid(output).item()

    # Calculate confidence for 'elephant' (class 0)
    # If prob_class_1 is near 0, it means it's definitely an elephant
    elephant_confidence = (1 - prob_class_1)

    if elephant_confidence > 0.5:
        label = "Elephant"
        display_conf = elephant_confidence
    else:
        label = "Not Elephant"
        display_conf = prob_class_1 # Confidence of it being NOT an elephant
```

```
    print(f"\nResult: {label}")
    print(f"Raw Confidence: {display_conf:.4f}")

    plt.imshow(img)
    plt.title(f"{label} ({display_conf*100:.1f}%)")
    plt.axis('off')
    plt.show()
```

```
Upload dataset.zip:
```

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving dataset.zip to dataset (11).zip
Detected Classes: {'elephant': 0, 'not_elephant': 1}

Training for high confidence...
Epoch 1/15 | Loss: 0.6774
Epoch 2/15 | Loss: 0.6888
Epoch 3/15 | Loss: 0.6701
Epoch 4/15 | Loss: 0.6143
Epoch 5/15 | Loss: 0.5852
Epoch 6/15 | Loss: 0.5705
Epoch 7/15 | Loss: 0.5214
Epoch 8/15 | Loss: 0.4897
Epoch 9/15 | Loss: 0.4657
Epoch 10/15 | Loss: 0.4527
Epoch 11/15 | Loss: 0.3998
Epoch 12/15 | Loss: 0.3934
Epoch 13/15 | Loss: 0.3579
Epoch 14/15 | Loss: 0.3464
Epoch 15/15 | Loss: 0.3359

Upload test image:
```

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving OIP (3).jpg to OIP (3).jpg

Result: Elephant
Raw Confidence: 0.7246
```

Elephant (72.5%)